

The Signal Grid

Electrothon — Electronics Club, Signal Processing Domain

Linga Raja
Roll No: 250121025

July 7, 2026

1 Problem Overview

Each of the five `cursed_schematic` images supplied with this problem has been corrupted in two independent ways:

1. **Salt-and-pepper noise** — isolated pixels randomly forced to pure black (0) or pure white (255).
2. **A repeating interference grid** — a periodic pattern laid over the whole image, obscuring the underlying blueprint.

The two corruptions are best removed in two different domains: the salt-and-pepper noise is spatially local and impulsive, so it is removed in the *spatial* domain with a median filter; the interference grid is periodic and global, so it shows up as a small number of sharp, isolated peaks in the *frequency* domain and is removed there with a notch filter. This report walks through both stages – the *Why*, *How*, and *Code* – and shows the resulting image after each step, using `cursed_schematic_01.png` as the running example. Section 6 then shows that the same, single script generalises cleanly to all five test images without any per-image tuning.

The full 5-panel output required by the evaluation criteria is shown in Figure 1.

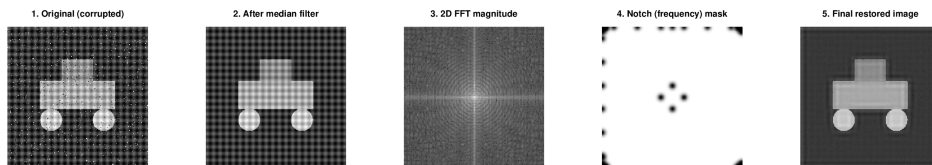


Figure 1: The required 5-panel visual output for `cursed_schematic_01.png`: original, after median filtering, FFT magnitude spectrum, notch mask, and the final restored image.

2 Part 1 — The Spatial Cleansing (Median Filter)

The Why

Salt-and-pepper noise consists of isolated pixels pushed to extreme values, completely unrelated to their neighbours. A linear smoothing filter (a mean or Gaussian blur) computes a *weighted average* of the neighbourhood, so a single corrupted pixel at value 255 still pulls the average upward – the noise gets spread into the surrounding pixels instead of being removed. A **median** filter instead sorts the neighbourhood and keeps the middle value. Since salt-and-pepper pixels sit at the very top or bottom of that sorted list, they are simply outvoted by the majority of uncorrupted neighbours and discarded entirely, while flat regions and edges are left intact.

The How

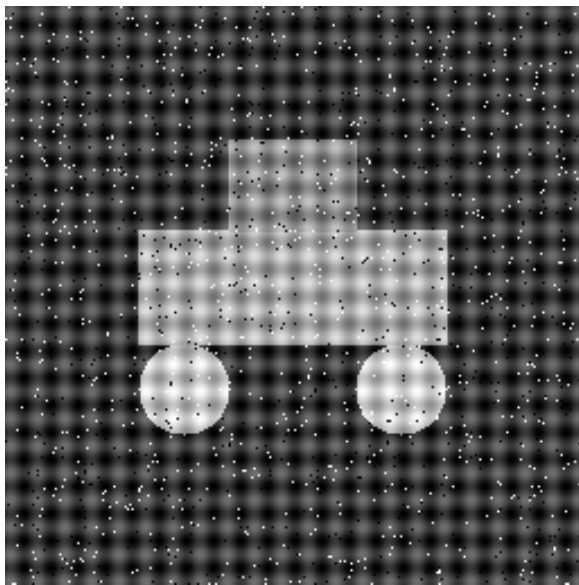
A 3×3 neighbourhood was tested against a 5×5 one. Both removed the impulse noise almost completely, but the 5×5 window visibly rounded off the sharp corners of the schematic's shapes (the circles and rectangles lose their crisp edges). The 3×3 window was therefore kept, as the better trade-off between denoising strength and preserving structural detail – exactly what the problem statement asks for.

The Code

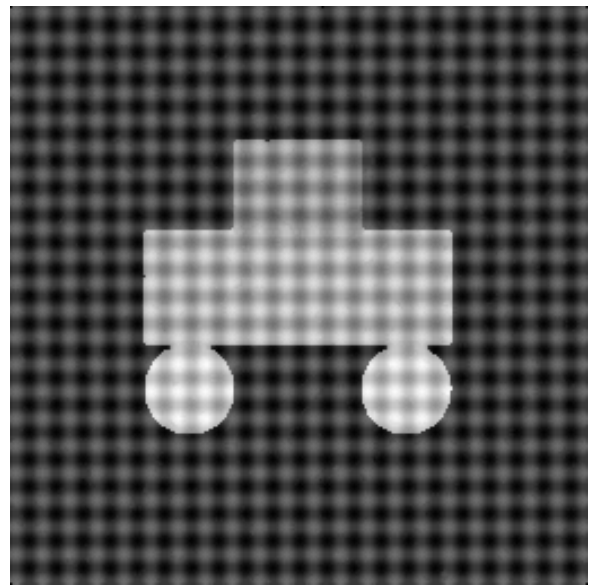
```
1 spatialCleaned = medfilt2(uint8(original), medianWindow);  
2 spatialCleaned = double(spatialCleaned);
```

`medianWindow` is set to `[3 3]`. `medfilt2` slides a 3×3 window over the image and, at every position, replaces the centre pixel with the median of the 9 pixels inside the window – directly implementing the idea described above.

Screenshot



Before: original corrupted image



After: 3×3 median filter

Figure 2: The salt-and-pepper speckle is essentially gone; the periodic grid (being a structured pattern, not an impulsive one) survives median filtering, exactly as expected – it is dealt with next, in the frequency domain.

3 Part 2 — The Frequency Exorcism (2D FFT)

The Why

The interference remaining after median filtering is a *periodic* pattern repeated regularly across the whole canvas. A repeating spatial pattern with some period concentrates almost all of its energy into a small number of specific frequencies – rather than chase it pixel-by-pixel in the spatial domain, transforming to the frequency domain turns the entire grid into a handful of bright, isolated dots that can be directly targeted and removed.

The How

The median-filtered image is transformed with a 2D Fast Fourier Transform, and `fftshift` is used to move the zero-frequency (DC) term to the centre of the array, so that radial distance from the centre corresponds directly to spatial frequency. The magnitude is viewed on a log scale purely because raw FFT magnitudes span an enormous range (the DC term dwarfs everything else), and the log compresses this so that the fainter interference peaks are visible alongside it.

```
1 F = fft2(spatialCleaned);  
2 Fshifted = fftshift(F);  
3 magSpectrum = log(1 + abs(Fshifted));
```

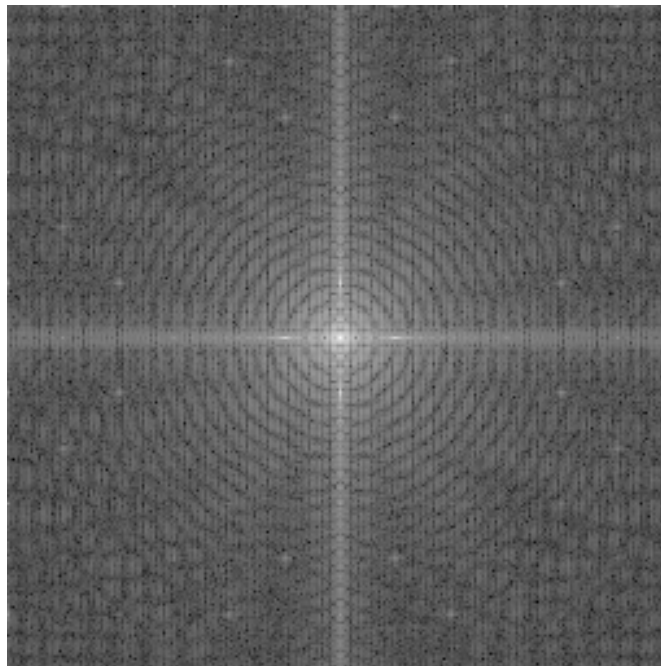


Figure 3: The 2D FFT magnitude spectrum. The bright centre is the low-frequency content of the schematic itself; the smaller bright dots scattered around it (best seen along the horizontal/vertical axes and the faint diagonal ring) are exactly the periodic grid noise.

Identifying the Noise Coordinates

Visually, the interference peaks are recognisable as sharp, isolated bright dots that sit away from the centre, whereas genuine image content produces a broad, smoothly decaying blob concentrated at the centre. That visual distinction is exactly what was turned into code (and is

also this submission's answer to the **Automated Peak Detection** bonus, Section 4): rather than reading pixel coordinates off the plot by eye and hardcoding them, the script detects them programmatically, which also makes it work unmodified on all five test images even though each has a differently spaced and oriented grid (compare the spectra in Figure 7).

4 Bonus: Automated Peak Detection

The Why

Hardcoded coordinates would only ever work for one specific grid spacing and orientation. The five sample images already have visibly different grids, so a fixed set of coordinates picked for one image would be wrong for the others – an adaptive detector is both more robust and directly reusable on images the pipeline has never seen.

The How

A naive approach – flag any spectrum pixel brighter than $(\text{mean} + k \times \text{std})$ of the *whole* spectrum – fails here, because a real image’s spectrum already falls off smoothly and strongly from the centre outward. Points just outside the protected central radius are naturally much brighter than points near the corners simply because they are lower-frequency, not because they are noise; a single global threshold either misses genuine peaks far from the centre or wrongly flags ordinary content close to it.

The fix is to compare every point to its *local* surroundings instead of a global number:

1. Estimate the smooth background trend of the spectrum with a wide median filter. A median filter is robust to sparse, sharp spikes, so its output is close to what the spectrum would look like *without* the interference.
2. Subtract that trend from the spectrum to get a residual that is close to flat everywhere except exactly at the interference spikes.
3. Flag a residual pixel as a genuine peak only if it is a local maximum (`imregionalmax`) *and* stands several standard deviations above the residual’s own statistics, *and* lies outside a small protected radius around the DC term (so the true low-frequency core is never touched).

The Code

```
1 background = medfilt2(magSpectrum, [bgWin bgWin]);
2 residual = magSpectrum - background;
3 ...
4 isRegionalMax = imregionalmax(residual);
5 candidatePeaks = isRegionalMax & outsideCore;
6
7 backgroundStats = residual(outsideCore);
8 peakThreshold = mean(backgroundStats) + peakStdThreshold * std(backgroundStats);
9 finalPeakMask = candidatePeaks & (residual > peakThreshold);
```

All of the thresholds above (`excludeRadius`, `bgWin`, `peakStdThreshold`) are set once, as fractions of the image size, at the top of the script – not per image – which is what lets the same run automatically detect 34 to 60 peaks across the five different test images (Table 1).

Building and applying the notch mask

Why a Gaussian notch, not a hard cutoff: an ideal (hard) notch filter has an abrupt edge in the frequency domain, and an abrupt edge in one domain always produces ringing (the

Gibbs phenomenon) in the other. A smooth Gaussian dip suppresses the same peak but decays gradually, which keeps the reconstructed image visually clean.

```

1 notchMask = ones(rows, cols);
2 for i = 1:numel(peakRows)
3     dip = notchStrength * exp( -((colGrid - peakCols(i)).^2 + (rowGrid - peakRows(i)).^2) ...
4                               / (2 * notchRadius^2) );
5     notchMask = notchMask .* (1 - dip);
6 end

```



Figure 4: The resulting notch mask: white = untouched (1.0), the small dark dots are the automatically-detected interference peaks, and the centre is always fully protected regardless of what the detector finds.

Inverse FFT and final restoration

The mask is applied multiplicatively in the frequency domain, then `ifftshift` undoes the earlier centring before the inverse FFT brings the image back to the spatial domain. Only the real part is kept (the imaginary part is, up to floating-point error, zero for a properly symmetric mask applied to a real image's spectrum).

```
1 Fmasked = Fshifted .* notchMask;  
2 reconstructed = real(ifft2(ifftshift(Fmasked)));  
3 restored = uint8(max(0, min(255, reconstructed)));
```

A notch filter can occasionally overshoot slightly above 255 or below 0 (mild Gibbs ringing). These are *clipped* back into the valid range rather than the whole image being rescaled, so the true brightness of the schematic is preserved instead of being artificially stretched.

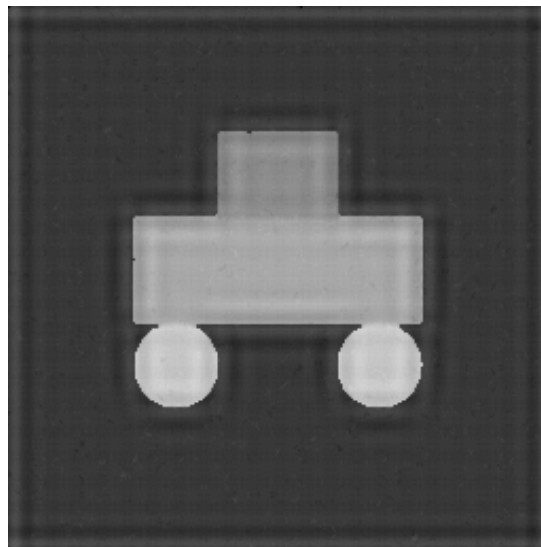


Figure 5: The final restored image: the interference grid is gone and the schematic (a simple vehicle shape) is clearly recovered.

5 Bonus: Blueprint Colour Mapping

The Why

This is a purely presentational bonus: classic engineering blueprints are white/cyan linework printed on a deep blue background, and mapping the restored grayscale image through a matching colour scale makes the final deliverable look the part.

The How

A custom 256-level colormap is built by linearly interpolating between a dark blue and a near-white colour. The restored image is contrast-stretched and converted to indices into that colormap.

The Code

```
1 blueDark = [0.03 0.15 0.38];  
2 blueLight = [0.90 0.97 1.00];  
3 blueprintMap = (1 - t) .* blueDark + t .* blueLight;  
4 ...  
5 restoredForColour = mat2gray(reconstructed);  
6 restoredIndexed = gray2ind(restoredForColour, nMapLevels);  
7 restoredBlueprint = ind2rgb(restoredIndexed, blueprintMap);
```

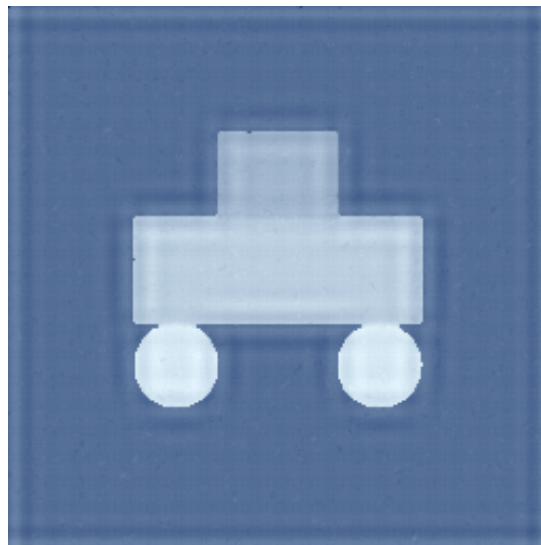


Figure 6: The restored image after the blueprint colour mapping.

6 Results Across All Five Test Images

The script places no per-image constants anywhere – every threshold is derived from the image’s own size and the statistics of its own spectrum – so it was run, unmodified, across all five supplied images. Table 1 summarises the outcome, using the standard deviation of a flat 40×40 background patch (top-left corner) as a simple proxy for how much noise remains: a lower number means a flatter, cleaner background.

Image	Peaks found	Std. before	Std. after	Reduction
cursed_schematic_01	47	38.50	7.81	79.7%
cursed_schematic_02	50	41.00	3.53	91.4%
cursed_schematic_03	34	44.59	9.37	79.0%
cursed_schematic_04	41	40.04	7.99	80.0%
cursed_schematic_05	60	42.17	8.36	80.2%

Table 1: Automatically-detected peak count and background noise level (standard deviation of a flat 40×40 patch) before/after restoration, for every supplied test image (`.png` extension omitted for space). All five show a 79–91% drop in background noise.

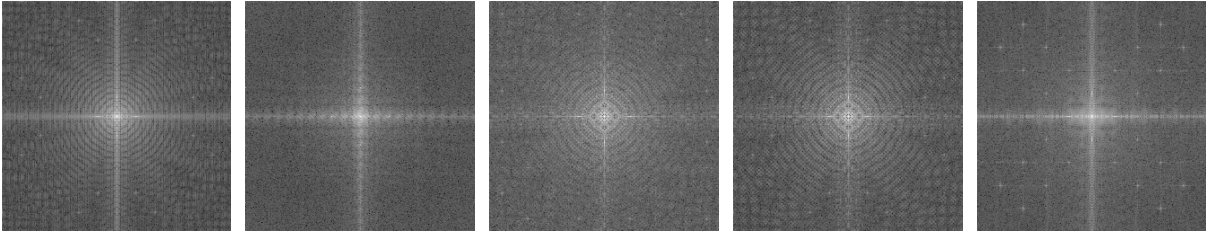


Figure 7: FFT magnitude spectra of all five test images (01–05, left to right). Each grid pattern has a visibly different spacing and orientation of peaks, which is exactly why an adaptive, automatic detector was necessary rather than one hardcoded set of coordinates.

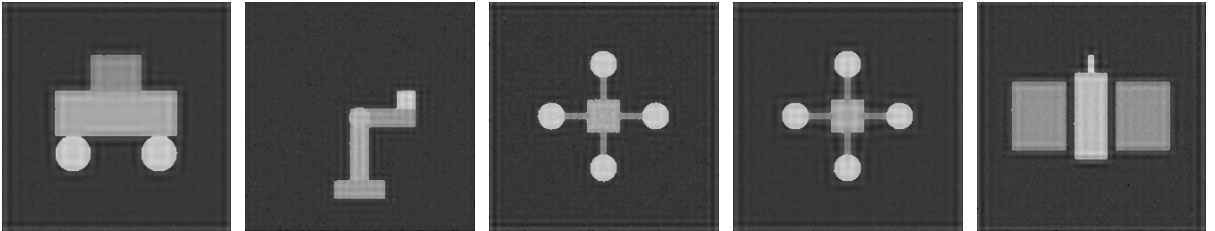


Figure 8: Final restored output for all five test images (01–05, left to right), produced by the same single, unmodified script.

7 Conclusion

The corruption in each schematic was removed in two stages, each matched to the domain where it is easiest to handle: a 3×3 median filter in the spatial domain for the impulsive salt-and-pepper noise, followed by an automatically-built Gaussian notch filter in the frequency domain for the periodic interference grid. Because every threshold in the pipeline is derived from the image's own size and spectrum statistics rather than hardcoded, one single script – with no per-image modification – recovers a clean schematic from all five supplied test images, and additionally produces a custom blueprint-styled version of each result.